Data Structures and Algorithms Lab
Assignment 3
Experiment 3: To demonstrate use of arrays and linked list to implement Stack operations and applications of Stack.

Instruction: Make a pdf file of the program code in C and its output.

Name: Chaitanya Sharma
SAP ID: 590026716
Batch: 39
Link of your GitHub Account:
https://github.com/Chaitanya189/Chaitanya_590026716_DSA/new/main
Course: B.Tech. CSE (Sem II)
Subject: DSA Lab
Date: 18 March 2026
Note: Do all the implementations in C programming.

1. Write a C program to implement a Stack data structure using an array. The program should be modular and use separate functions for each operation.
   (i) Insert (Push): Add an element to the top of the stack.
   (ii) Delete (Pop): Remove an element from the top of the stack.
   (iii) Display: Display all elements of the stack from top to bottom.

```c
// Q1: Stack using Array
#include <stdio.h>
#define MAX 100   // Maximum size of stack
int stack[MAX];
int top = -1;
 void push(int value) {
    if (top == MAX - 1) {
        printf("Stack is FULL (Overflow)!\n");
    } else {
        top++;
        stack[top] = value;
        printf("%d pushed into stack.\n", value);
    }
}
 void pop() {
    if (top == -1) {
        printf("Stack is EMPTY (Underflow)!\n");
    } else {
        printf("%d popped from stack.\n", stack[top]);
        top--;
    }
}
 void display() {
    if (top == -1) {
        printf("Stack is EMPTY.\n");
    } else {
        printf("Stack elements (top to bottom): ");
        for (int i = top; i >= 0; i--) {
            printf("%d ", stack[i]);
        }
        printf("\n");
    }
}
int main() {
    int choice, value;
    while (1) {
        printf("\n--- Stack Menu ---\n");
        printf("1. Push\n");
        printf("2. Pop\n");
        printf("3. Display\n");
        printf("4. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter value to push: ");
                scanf("%d", &value);
                push(value);
                break;
            case 2:
                pop();
                break;
            case 3:
                display();
                break;
            case 4:
                return 0;
            default:
                printf("Invalid choice!\n");
        }
    }
    return 0;
}
```

2. Write a C program to implement a Stack using a Linked List. The program should support the following operations using separate functions:

(i) Push (Insert): Insert an element into the stack.
(ii) Pop (Delete): Remove an element from the stack.
(iii) Display: Display all elements of the stack.

```c
// Q2: Stack using Linked List
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node* next;
};
struct Node* top = NULL;
void push(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = top;
    top = newNode;
    printf("%d pushed into stack.\n", value);
}
void pop() {
    if (top == NULL) {
        printf("Stack is EMPTY (Underflow)!\n");
    } else {
        struct Node* temp = top;
        printf("%d popped from stack.\n", top->data);
        top = top->next;
        free(temp);
    }
}
void display() {
    if (top == NULL) {
        printf("Stack is EMPTY.\n");
    } else {
        struct Node* temp = top;
        printf("Stack elements (top to bottom): ");
        while (temp != NULL) {
            printf("%d ", temp->data);
            temp = temp->next;
        }
        printf("\n");
    }
}
int main() {
    int choice, value;
    while (1) {
        printf("\n--- Stack Menu ---\n");
        printf("1. Push\n");
        printf("2. Pop\n");
        printf("3. Display\n");
        printf("4. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter value to push: ");
                scanf("%d", &value);
                push(value);
                break;
            case 2:
                pop();
                break;
            case 3:
                display();
                break;
            case 4:
                return 0;
            default:
                printf("Invalid choice!\n");
        }
    }
    return 0;
}
```

3. Write a C program to reverse a given string using a stack. The program should follow the **Last In First Out (LIFO)** principle of the stack.

```c
// Q3: Reverse a String using Stack (LIFO principle)
#include <stdio.h>
#include <string.h>
#define MAX 100
char stack[MAX];
int top = -1;
void push(char ch) {
    top++;
    stack[top] = ch;
}
char pop() {
    char ch = stack[top];
    top--;
    return ch;
}
int main() {
    char str[MAX];
    printf("Enter a string: ");
    scanf("%s", str);

    int len = strlen(str);
    for (int i = 0; i < len; i++) {
        push(str[i]);
    }
    printf("Reversed string: ");
    for (int i = 0; i < len; i++) {
        printf("%c", pop());
    }
    printf("\n");
    return 0;
}
```

4. Write a program to check whether a given expression is **balanced or not** using a **stack implemented with an array**.

```c
// Q4: Check if an expression has Balanced Brackets using Stack
#include <stdio.h>
#include <string.h>
#define MAX 100
char stack[MAX];
int top = -1;
void push(char ch) {
    top++;
    stack[top] = ch;
}
char pop() {
    if (top == -1) return '\0';
    char ch = stack[top];
    top--;
    return ch;
}
int isBalanced(char expr[]) {
    int len = strlen(expr);

    for (int i = 0; i < len; i++) {
        char ch = expr[i];
        if (ch == '(' || ch == '[' || ch == '{') {
            push(ch);
        }
        else if (ch == ')' || ch == ']' || ch == '}') {
            if (top == -1) {
                return 0;
            }
            char top_ch = pop();
            if ((ch == ')' && top_ch != '(') ||
                (ch == ']' && top_ch != '[') ||
                (ch == '}' && top_ch != '{')) {
                return 0;
            }
        }
    }
    return (top == -1) ? 1 : 0;
}
int main() {
    char expr[MAX];
    printf("Enter an expression: ");
    scanf("%s", expr);
    if (isBalanced(expr)) {
        printf("Expression is BALANCED.\n");
    } else {
        printf("Expression is NOT balanced.\n");
    }
    return 0;
}

```

5. Write a C program to convert a given **infix expression** (A+B*(C^D-E)^(F+G*H)-I) into its corresponding **postfix expression** (ABCD^E-FGH*+^*+I-) using a stack implemented with arrays.

```c
// Q5: Convert Infix Expression to Postfix using Stack
#include <stdio.h>
#include <string.h>
#define MAX 100
char stack[MAX];
int top = -1;
void push(char ch) {
    top++;
    stack[top] = ch;
}
char pop() {
    if (top == -1) return '\0';
    char ch = stack[top];
    top--;
    return ch;
}
char peek() {
    if (top == -1) return '\0';
    return stack[top];
}
int precedence(char op) {
    if (op == '^') return 3;
    if (op == '*' || op == '/') return 2;
    if (op == '+' || op == '-') return 1;
    return 0;
}
int isOperand(char ch) {
    return (ch >= 'A' && ch <= 'Z') ||
           (ch >= 'a' && ch <= 'z') ||
           (ch >= '0' && ch <= '9');
}
int isRightAssociative(char op) {
    return op == '^';
}
```

```c
void infixToPostfix(char infix[], char postfix[]) {
    int j = 0;
    for (int i = 0; infix[i] != '\0'; i++) {
        char ch = infix[i];
        if (isOperand(ch)) {
            postfix[j++] = ch;
        }
        else if (ch == '(') {
            push(ch);
        }
        else if (ch == ')') {
            while (peek() != '(') {
                postfix[j++] = pop();
            }
            pop();
        }
        else {
          while (top != -1 && peek() != '(' &&
                  (precedence(peek()) > precedence(ch) ||
                  (precedence(peek()) == precedence(ch) && !isRightAssociative(ch)))) {
                postfix[j++] = pop();
            }
            push(ch);
        }
    }
    while (top != -1) {
        postfix[j++] = pop();
    }

    postfix[j] = '\0';
}
int main() {
    char infix[MAX], postfix[MAX];
    printf("Enter infix expression: ");
    scanf("%s", infix);
    infixToPostfix(infix, postfix);
    printf("Postfix expression: %s\n", postfix);
    return 0;
}
```

6. Write a C program to **evaluate a postfix expression** using a **stack implemented with an array**.
   (i) Postfix Expression: 23*54*+9-
   (ii) Result = 17

```c
// Q6: Evaluate a Postfix Expression using Stack
#include <stdio.h>
#include <string.h>
#define MAX 100
int stack[MAX];
int top = -1;
void push(int value) {
    top++;
    stack[top] = value;
}
int pop() {
    int value = stack[top];
    top--;
    return value;
}
int evaluatePostfix(char expr[]) {
    int len = strlen(expr);
    for (int i = 0; i < len; i++) {
        char ch = expr[i];
        if (ch >= '0' && ch <= '9') {
            push(ch - '0');
        }
        else if (ch == '+' || ch == '-' || ch == '*' || ch == '/') {
            int b = pop();
            int a = pop();
            int result;
            if (ch == '+') result = a + b;
            else if (ch == '-') result = a - b;
            else if (ch == '*') result = a * b;
            else result = a / b;
            push(result);
        }
    }
    return pop();
}
int main() {
    char expr[MAX];
    printf("Enter postfix expression: ");
    scanf("%s", expr);
    int result = evaluatePostfix(expr);
    printf("Result = %d\n", result);
    return 0;
}
```